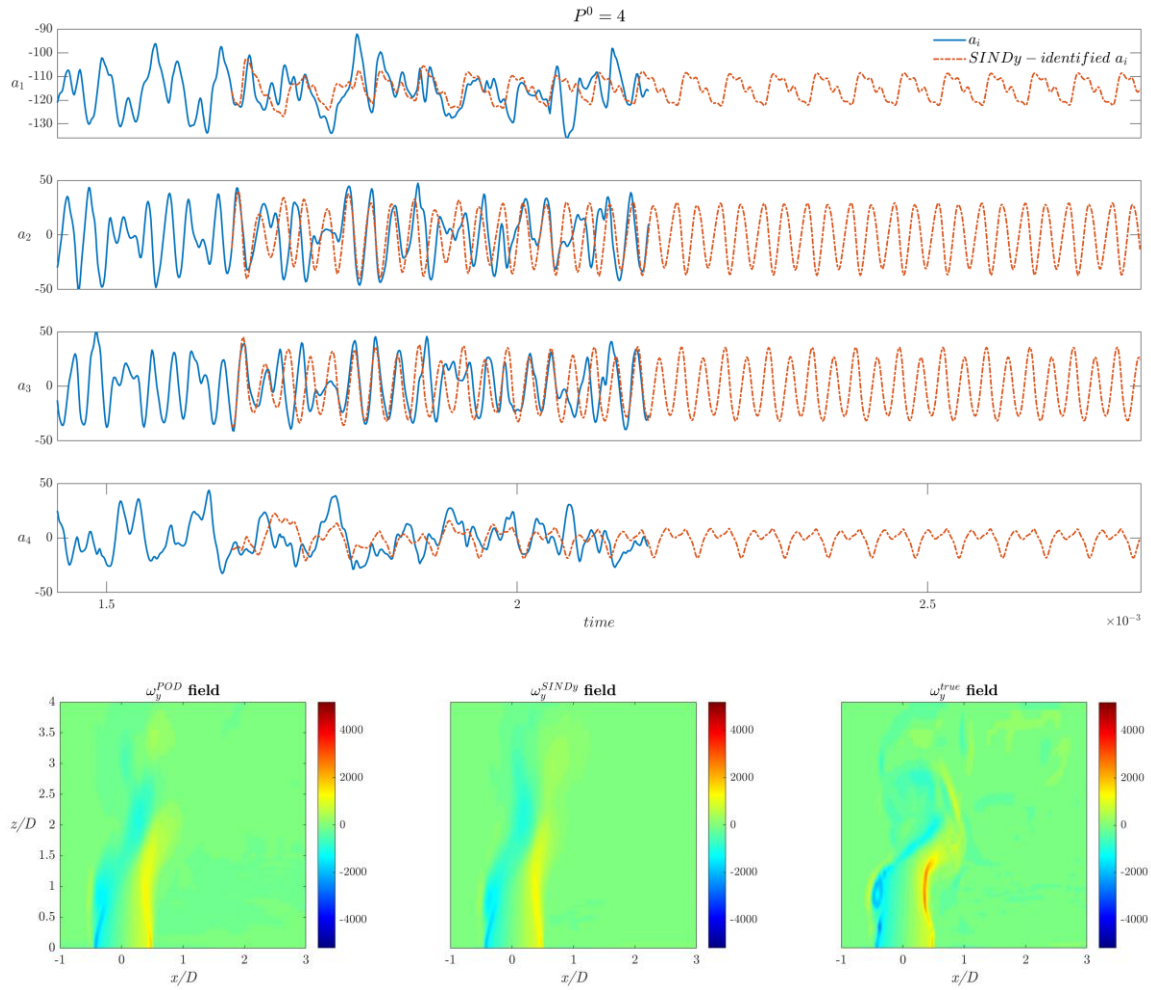


RASGAN for Flow-Field ‘Super-Resolution’: Model, Losses, and Training Logic

Background

From the data driven framework for design optimization using SINDy, POD coefficients are learned and predicted enabling the extension of simulation fields beyond simulation time. This is however of limited use as turbulent fields are not as wholly represented by large coherent structures unlike laminar fields hence POD reconstructed fields lack the smaller scale effects.



For the vorticity field that means the POD reconstructed field fails to capture the vortex cores. As such I'm proposing a complementary Neural Network framework to SINDy reconstructed fields in the pursuit of design optimization in a parameter space. From a quick literature search articles like “Super-resolution reconstruction of turbulent velocity fields using a GAN-based AI framework” [1] and “Influence of adversarial training on super-resolution turbulence reconstruction” [16] already implemented Super-Resolution GANs with simulation fields hence the GAN framework was chosen.

General GAN Framework

Input data is a “highest-energy-mode” POD reconstructed field of $H_2 - \text{Air}$ jet in crossflow simulation. First four modes used in reconstruction. The reference fields used are Pressure (P), $z -$ normal velocity (v), and vorticity (ω_v). The dataset is preprocessed with the mean (μ) and standard deviation (σ) of the true field training only (validation set not included) data by

$$\frac{x - \mu}{\sigma}$$

The Neural Network learns to map the POD reconstructed field to the true field. Three-channel data is used since the SRGAN basis, originally for image Super-Resolution with RGB inputs, was designed for 3 channels. The model conditions the discriminator on both the generated/true fields and the POD input so ‘realism’ is judged in context of the input not in a vacuum.

RASGAN and SRGAN/ESRGAN

SRGAN

Baseline model used is an SRGAN model obtained from a git repository^[3] implemented in TensorLayerX^[2]. TensorLayerX is a useful framework because it supports implementation of models, training, etc for different Neural Network backends such as Tensorflow and PyTorch.

- Generator: classic SRGAN residual blocks (RB; Conv2D stack) with BatchNormalization layers and PreLU activation.
- Discriminator: deep CNN with BatchNormalization; non-relativistic objective.
- Training: two stages; content pretrain with MSE/content, then adversarial with Binary Cross-Entropy (BCE) + VGG perceptual loss.

ESRGAN

Modified SRGAN model to target relativistic loss^[4] and reduce noise by removing BatchNormalization layers in line with Enhanced SRGAN^[6] (ESRGAN).

- Generator: Residual-in-Residual blocks^[1] (RRDB; denser Conv2D with skip structure using concatenation layers) with LeakyReLU activation.
- Discriminator unchanged from SRGAN.
- Training: pretrain stage unchanged, adversarial stage keeps VGG perceptual/content loss but uses relativistic logits fed to BCE.

RASGAN

Iteratively modified conditional GAN^[7] with ESRGAN Generator and patch-based conditional Discriminator^[8] to improve model performance. Custom losses also implemented to target POD $\rightarrow$ True fields.

- Generator: RRDB (with dilations), ECA ^[9] (light channel attention) layers and per-channel output heads for P , v and ω_y .
- Discriminator: Spectrally normalized ^[11] conditional PatchGAN, with SE ^[10] (Squeeze and Excitation) blocks, that sees generated/real, the POD input, residuals (true/fake – POD), and learned edge maps with a global head.
- Training: pretrain switched to MAE, VGG loss removed for both stages.
- Losses: robust pixel (balanced Charbonnier) ^[12], gradient (H^1) ^{[14][6]}, spectrum-aware losses (log-amplitude + energy spectrum) ^[15], residual alignment, low-k consistency, TV, feature matching ^[13], relativistic average GAN (RA-SGAN) with softplus activation, R1 regularization ^[17], instance noise, and two physics-motivated decorrelation terms (edge alignment penalty & off-diagonal covariance).

Network Architecture and Training Structure

Model Architecture

Generator

Conv2D and Swish (SeLU) activation layer, followed by RRDB with Swish activation and symmetric dilations.

- Efficient channel attention (ECA) on residual branches.
- Shared trunk fans out into three per-channel heads (light residual heads + 1×1 Conv2D) to predict P , v , ω_y separately.
- Optional physical coordinate map can be concatenated to inputs.

Discriminator

- Conditional input packs 12 channels: the candidate field ('fake' or 'real'), the POD input, 'fake'/'real' residuals ('fake'/'real' - POD), and learned edge maps per channel.
- Optional conditioning dropout and solo-channel passes make D focus on one physical channel at a time.
- Spectral normalization on convolution/linear layers, Squeeze-and-Excitation early on, a patch head plus a global head (broadcast and added to patch logits).

Loss Functions

Notation:

$\mathbf{x}$ = generated tensor (fake); $\mathbf{y}$ = ground-truth tensor (real); $\mathbf{r} = (\mathbf{x} - \mathbf{y})$ (residual error)

$\mathbf{l}$ = POD input; tensors are $\mathbf{N} \times \mathbf{C} \times \mathbf{H} \times \mathbf{W}$.

Balanced Charbonnier (Robust Pixel Loss)

Goal: robust per-channel fit without letting a single channel's scale dominate. For each channel c , estimate a robust scale $s_c = \text{mean}_{n,i,j} |y_{n,c,i,j}|$. Normalize residual error using the scale.

loss definition:

$$L_{charb} = \frac{1}{NCHW} \sum_{n,c,i,j} w_c^{bal} \cdot w_c (r_{n,c,i,j}^2 + \varepsilon^2)^{\frac{\alpha}{2}}$$

Gradient (H') Loss

Goal: match edge strength and orientation. Using central differences ∇_x, ∇_y and the same balanced Charbonnier core $C(\cdot)$:

$$L_{grad} = \lambda_{grad} [C(\nabla_x x, \nabla_x y) + C(\nabla_y x, \nabla_y y)]$$

Residual Alignment & Low-k Consistency

Goal: in the input-true residual space $(x - l)$, match the true correction $(y - l)$ and emphasize large-scale (low-k) structure.

$$L_{res} = \lambda_{res} \cdot C(\text{crop}(x - l), \text{crop}(y - l))$$

$$L_{lowk} = \lambda_{lowk} \cdot \| |\log|F(x - l)| | \times \text{LOW}_{\text{MASK}} \|_1$$

where F is rFFT2D of a Hann-windowed crop; LOW_{MASK} selects low radial wavenumbers.

Log-Amplitude Spectrum Loss

Goal: match the log amplitude of the residual's 2D spectrum (shape across scales), normalized per sample/channel. Let

$$\text{LA}_x = \log |F(x - l)|$$

and (μ, σ) are per-sample, per-channel of LA_y . Then:

$$L_{spec} = \lambda_{spec} \cdot E \left[\left| \frac{\text{LA}_x - \mu}{\sigma} - \frac{\text{LA}_y - \mu}{\sigma} \right| \right]$$

Energy Spectrum (Radial) Loss

Goal: match the radially averaged energy spectrum with gentle high-k emphasis. Define energy $E = \exp^{(2 \cdot LA)}$ with bin masks M_b over (k_x, k_y) , per-bin averages $S_x[b]$ and $S_y[b]$ (standardized by μ and σ),

$$L_{\text{espec}} = \lambda_{\text{espec}} \cdot \left(\frac{1}{B}\right) \sum_b w_b \cdot \left| \frac{((S_x[b] - \mu_x) - (S_y[b] - \mu_y))}{\sigma_y} \right|$$

Feature Matching (FM)

Goal: a VGG-free “perceptual” signal. Match features from the last D blocks:

$$L_{\text{fm}} = \lambda_{\text{fm}} \cdot \sum_{\{k \in \{-3, -2, -1\}\}} C(f_k(x), f_k(y))$$

Total Variation (TV)

Goal: damp grid-scale noise:

$$L_{\text{tv}} = \lambda_{\text{tv}} \left(\|\nabla_x x\|_2^2 + \|\nabla_y y\|_2^2 \right)$$

Cross-Channel Edge Decorrelation

Goal: prevent pressure edges from bleeding into velocity/vorticity (and vice-versa).

For gradients (g_a, g_b) of channel pairs, penalize squared cosine alignment weighted by edge strength:

$$L_{\text{decor}} = \lambda_{\text{decor}} \cdot E[w \cdot \cos^2 \theta]$$

with

$$w = \tanh(\gamma \|g_a\|) \cdot \tanh(\gamma \|g_b\|)$$

Off-Diagonal Covariance Penalty

Goal: reduce spurious cross-channel correlation. Using either output channels or their gradient magnitudes. Form the batch correlation matrix R per sample and penalize off-diagonal energy:

$$L_{\text{cov}} = \lambda_{\text{cov}} \cdot E[\|R - \text{diag}(R)\|_F^2]$$

Adversarial (Relativistic Average SGAN, Softplus)

Goal: make ‘fake’ look more real than ‘real’ on average. With discriminator patch logits $D(\cdot)$, batch means $\bar{r} = E[D(y)]$ and $\bar{f} = E[D(x)]$:

$$L_D^{RA} = E[\text{softplus}(-(D(y) - \bar{f}))] + E[\text{softplus}(D(x) - \bar{r})]$$

And

$$L_G^{RA} = E[\text{softplus}(-(D(x) - \bar{r}))] + E[\text{softplus}(D(y) - \bar{f})]$$

Weighted by a scheduled λ_{adv} , where

$$\text{softplus}(z) = \log(1 + e^z)$$

R1 Gradient Penalty (on Real, Lazy)

Goal: stabilize D and set a meaningful logit scale. On the real D -input (inp), which packs y , l , r and edges (12 channels); let s be the spatial mean of $D(\text{inp})$. Then:

$$L_{R1} = \frac{\gamma}{2} \cdot E[\| \nabla_{\text{inp}} s \|^2_2]$$

applied some predetermined number of steps and scaled accordingly.

Instance Noise

Goal: blur D ’s decision boundary early on. Add $N(0, \sigma^2)$ to both real and fake inputs to D with a small σ that decays over training.

Total Generator Loss (Adversarial Stage)

$$L_G = L_{charb} + L_{grad} + L_{res} + L_{lowk} + L_{spec} + L_{espec} + L_{tv} + L_{fm} + L_{decor} + L_{cov} + \lambda_{adv} \cdot L_G^{RA}$$

During content pretraining, only the content stack is used—no adversarial, FM, or heavy spectral terms.

Training Logic

Stages

- Content pretrain with MAE based early stopping; generator only, using
$$L_G = L_{charb} + L_{grad} + L_{res} + L_{lowk}$$
- Adversarial (up to 300 epochs): alternate G/D with all loss terms enabled

Learning-Rate (LR) & Weight Steering

- G LR constrained to $[5e - 7, 5e - 4]$; D LR constrained to $[1e - 7, 1e - 5]$ (Two Time-Scale Update Rule) ^[18]
- A steerer monitors behavioral and validation signals and adjusts LR up/down with $(\times 1.2 / \times 0.4)$ factor respectively with plateau patience and early-stop safeguards.
- Adversarial weight scheduling: λ_{adv} warms from a tiny value (ADV_{INIT}) and stays within $[ADV_{MIN}, ADV_{MAX}]$ with a short patience and cooldown.
- Regularizers over time: R1 ramps in $(0 \rightarrow \gamma_{base})$, instance noise decays, conditioning dropout anneals, edge emphasis can anneal.
- Spectrum ramp: spectral loss weights can ramp in over approximately 40 epochs.

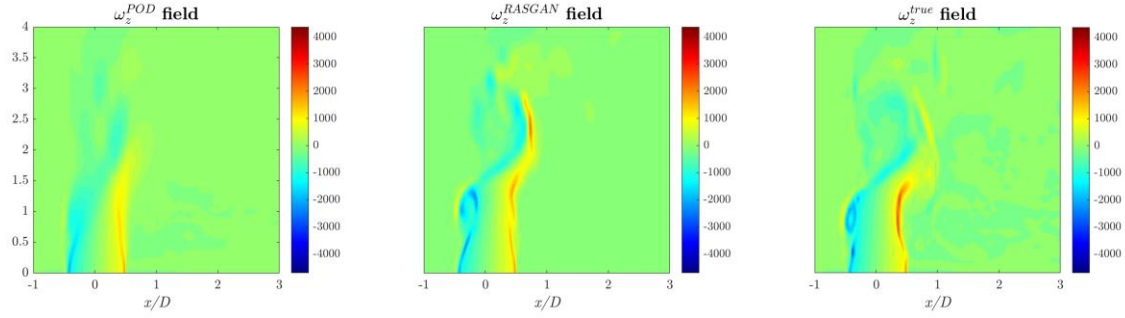
Validation, EMA, and Checkpoints

- Validation runs on the current G; an Exponential Moving Average (EMA) ^[19] shadow of G can be maintained, and both raw/EMA weights can be saved.
- Checkpoints include a ‘meta’ with input shapes, current epoch, LR state, steerer/penalty weights, and which family (raw vs. EMA) validated best to enable restoring train state to resume/restart. It also ensures inference uses the best model.

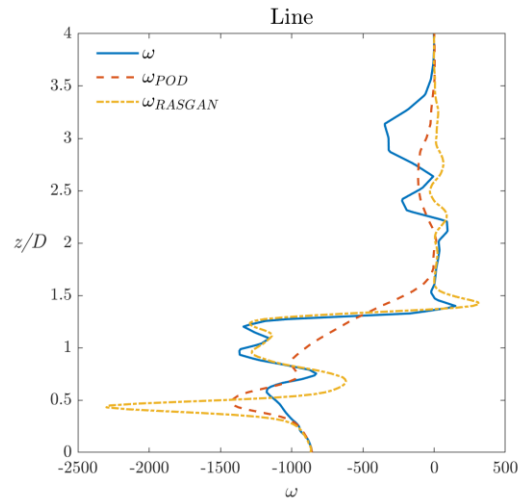
Results

RASGAN Preliminary Results

2D contour

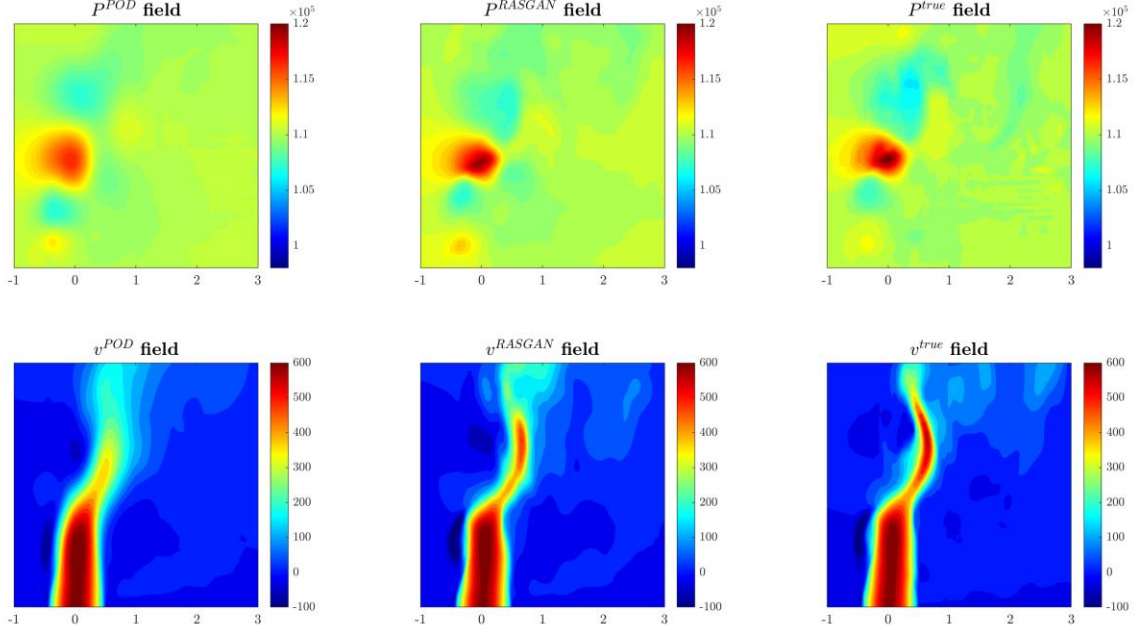


Line plot



The RASGAN model does to some extent add back the small scale effect in fields. For the vorticity field the vortex core (contour plot) is captured compared to the pod reconstructed field. The line plot also shows how the RASGAN prediction better fits the true field but also exacerbates an error that was originally present in the POD field.

Other Channels

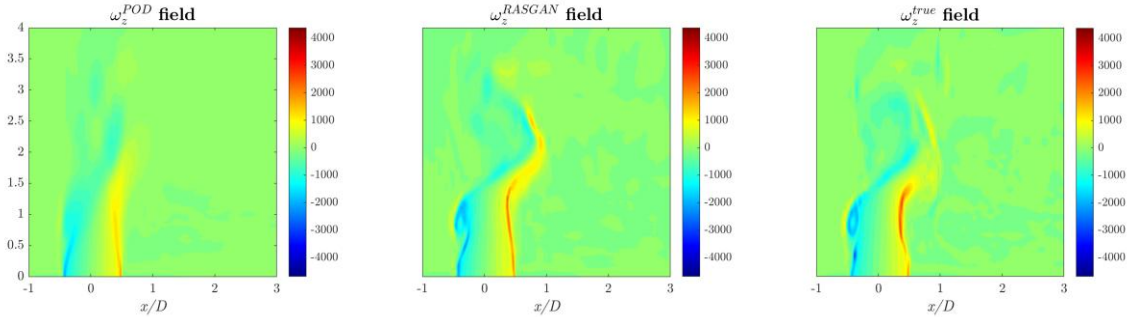


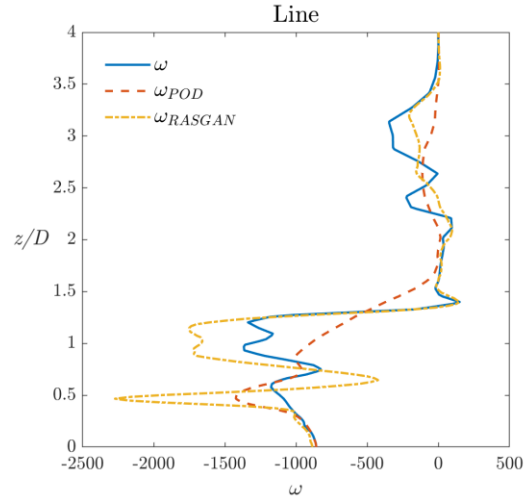
The RASGAN velocity field better captures the trail while the pressure prediction is a mild improvement on the POD reconstruction.

Transformer based Generator

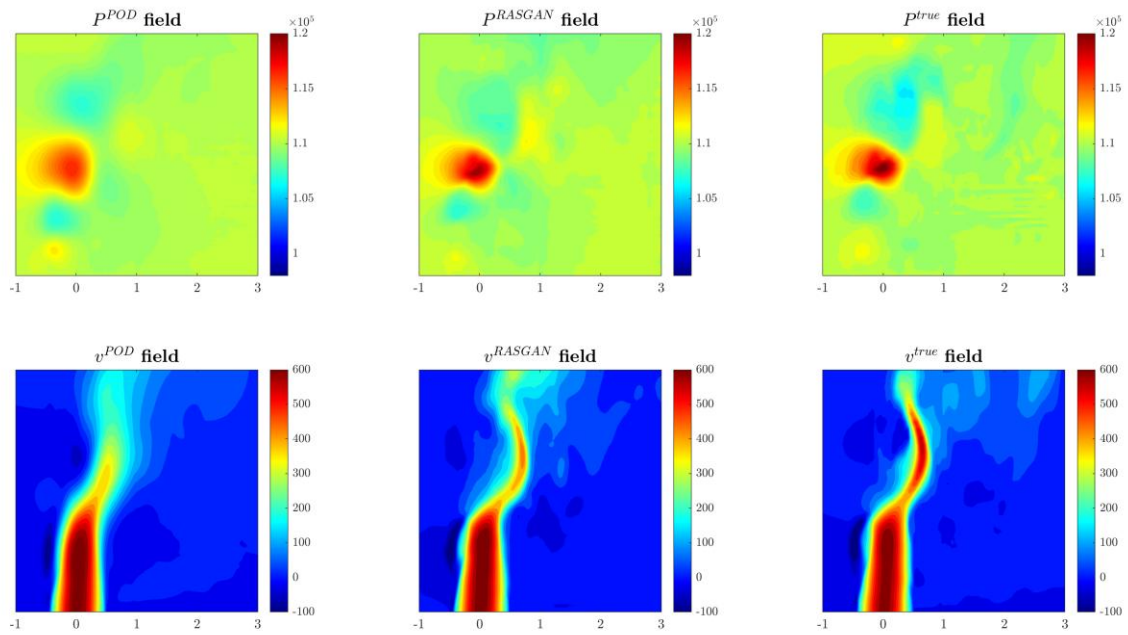
TRANSKAN

In line with the prevalence of Transformer based models, due to their advantages over CNNs for data with long range dependencies ^[20], a Transformer based generator is being considered. The Transformer generator has two input layers, the first being the (N, C, H, W) input (same as RASGAN model) and the second being an $(N, 12)$ input of the four (4) POD coefficients (per channel) used in reconstruction. Here are preliminary results of the TRANSKAN model





The overall field is better captured by the TRANSGAN model. The general trend of the line plot is captured but more erroneous than the RASGAN model.



Same as with the vorticity field. The overall field is more accurate and for velocity the trail is better resolved than the RASGAN model.

References

- [1] Fukami, K., Nabae, Y., Kawai, K., & Fukagata, K. “*Super-resolution reconstruction of turbulent velocity fields using a GAN-based AI framework*”, <https://doi.org/10.1063/1.5127031>
- [2] TensorLayerX framework: <https://github.com/tensorlayer/TensorLayerX#installation>
- [3] TensorLayer SRGAN repository: <https://github.com/tensorlayer/SRGAN/tree/master>
- [4] Jolicœur-Martineau, A. “*The relativistic discriminator: a key element missing from standard GAN*”, <https://doi.org/10.48550/arXiv.1807.00734>
- [5] “*Single MR Image Super-Resolution using Generative Adversarial Network*”, <https://doi.org/10.48550/arXiv.2207.08036>
- [6] Xintao Wang et al. “*ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks*”, <https://doi.org/10.48550/arXiv.1809.00219>
- [7] Mehdi Mirza, Simon Osindero, “*Conditional Generative Adversarial Nets*”, <https://doi.org/10.48550/arXiv.1411.1784>
- [8] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, Alexei A. Efros, “*Image-to-Image Translation with Conditional Adversarial Networks*”, <https://doi.org/10.48550/arXiv.1611.07004>
- [9] Qilong Wang et al. “*ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks*”, <https://doi.org/10.48550/arXiv.1910.03151>
- [10] Jie Hu et al. , “*Squeeze-and-Excitation Networks*”, <https://doi.org/10.48550/arXiv.1709.01507>
- [11] Takeru Miyato et al., “*Spectral Normalization for Generative Adversarial Networks*”, <https://doi.org/10.48550/arXiv.1802.05957>
- [12] Wei-Sheng Lai et al., “*Deep Laplacian Pyramid Networks for Fast and Accurate Super-Resolution*”, <https://doi.org/10.48550/arXiv.1704.03915>
- [13] Tim Salimans et al. “*Improved Techniques for Training GANs*”, <https://doi.org/10.48550/arXiv.1606.03498>
- [14] Jianing et al., “*Focal Frequency Loss for Image Reconstruction and Synthesis*”, <https://doi.org/10.48550/arXiv.2012.12821>
- [15] Bode et al., “*Using physics-informed enhanced super-resolution generative adversarial networks for subfilter modeling in turbulent reactive flows*”, <https://doi.org/10.1016/j.proci.2020.06.022>
- [16] Bode et al., “*Influence of adversarial training on super-resolution turbulence reconstruction*”, <https://doi.org/10.1103/PhysRevFluids.9.064601>
- [17] Mescheder et al., “*Which Training Methods for GANs do actually Converge*”, <https://doi.org/10.48550/arXiv.1801.04406>
- [18] Heusel et al., “*GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*”, <https://doi.org/10.48550/arXiv.1706.08500>
- [19] Morales-Brotons et al., “*Exponential Moving Average of Weights in Deep Learning: Dynamics and Benefits*”, <https://doi.org/10.48550/arXiv.2411.18704>
- [20] Yao et al., “*TM-SRGAN: image super-resolution reconstruction algorithm combining lightweight transformer and SRGAN*”, <https://doi.org/10.1117/1.JEI.34.3.033052>